



Escuela de Ingeniería y Ciencias
Ingeniería en Ciencia de Datos y Matemáticas
Bloque MA2006B Uso de Álgebras Modernas para Seguridad y Criptografía

Documentación de Desarrollo / SDK

Gestión de Identidades y Documentos v1.0

Equipo 3 — Grupo 601

Estudiante	Matrícula
Daniel Eugenio González Limas	A01285898
Sebastián Zaragoza Díaz	A01286211
Cedrick Patricio Treviño Ortiz	A01198868
Jorge Eduardo González Cantú	A01571510
André Robles Bueckmann	A01706832

Profesores: Raúl Gómez, Daniel Otero y Alberto F. Martínez

Socio Formador: Casa Monarca. Ayuda Humanitaria al Migrante, A.B.P.
Monterrey, Nuevo León. 14 de junio de 2026.

Índice de Contenidos

1. Descripción General.....	3
2. Arquitectura del Sistema.....	3
3. Stack Tecnológico.....	4
4. Estructura del Proyecto.....	4
5. Configuración del Entorno.....	5
6. Instalación y Ejecución.....	6
7. Base de Datos.....	6
8. API / Interfaces (núcleo del SDK).....	7
8.1 Autenticación y sesión.....	7
8.2 Segundo factor (TOTP y WebAuthn).....	8
8.3 Documentos.....	8
8.4 Administración (rol admin, salvo invitaciones).....	8
8.5 Invitaciones (admin y coordinator).....	9
8.6 Interfaz del frontend (SDK cliente).....	10
8.6.1 `api.ts` — núcleo de transporte.....	10
8.6.2 `csrf.ts` y `auth.ts` — sesión y autenticación.....	10
8.6.3 `webauthn.ts` — puente con el autenticador del dispositivo.....	11
8.6.4 `documents.ts` — gestión documental.....	11
8.6.5 Otros módulos.....	12
8.6.6 Ejemplo de uso end-to-end (firmar una revisión).....	13
9. Flujos y Lógica de Negocio.....	13
10. Ejemplos de Uso.....	13
11. Manejo de Errores.....	14
12. Pruebas.....	14
13. Despliegue.....	14
14. Seguridad.....	15
15. Guía de Contribución.....	15
16. Problemas Conocidos.....	16
17. FAQ Técnica.....	16
18. Referencias.....	16

1. Descripción General

Propósito del sistema. Casa Monarca Access Control es un **gestor de identidades y control de acceso** para gestionar documentos confidenciales de Casa Monarca. Permite al personal autorizado autenticarse con doble factor, subir y versionar documentos sensibles, firmarlos digitalmente con no-repudio y mantener una bitácora auditable de todas las acciones, con permisos diferenciados por rol.

Alcance.

- *Qué hace:* registro por invitación verificada, autenticación multifactor (TOTP + passkeys WebAuthn/FIDO2), RBAC de cuatro niveles, gestión documental con versionado y hash de integridad, firma digital, flujos de aprobación y auditoría completa.
- *Límites actuales:* el control de versiones (VCS) completo, el encadenamiento criptográfico de la bitácora y el endurecimiento de la atestación WebAuthn quedan como trabajo futuro. Despliegue probado en staging, no en producción final.

Casos de uso principales.

1. Un admin invita a un colaborador; tras verificación fuera de banda se emite el enlace de registro.
2. El colaborador se registra, define contraseña y enrola su segundo factor.
3. Un voluntario sube documentos; un no-coordinador los revisa/verifica; un coordinador sube nuevas revisiones y firma; un admin aprueba, gestiona usuarios y consulta auditoría.

Acción	Voluntario	No-coord.	Coordinador	Admin
Subir documento	✓	✓	✓	✓
Ver / descargar / verificar documentos	—	✓	✓	✓
Subir nueva revisión	—	—	✓	✓
Firmar revisión	—	—	✓	✓
Crear / gestionar invitaciones	—	—	✓	✓
Aprobar / rechazar documentos	—	—	—	✓
Gestionar usuarios, roles y recuperación	—	—	—	✓
Borrar documento (deja tombstone)	—	—	—	✓
Auditoría y libro de firmas	—	—	—	✓

Tabla 2. Matriz de permisos por rol (RBAC) (elaboración propia).

2. Arquitectura del Sistema

Tipo de arquitectura. Cliente-servidor **por capas** bajo un modelo de **mismo origen**: Apache sirve el frontend estático en / y hace proxy a la API en /api. El backend sigue el patrón Rutas → Controllers → Services → Models → BD.

Componentes principales.

- **Frontend** (Access-Control-CM-Web): SPA React/TypeScript.
- **Backend** (Access-Control-CM-API): API REST Laravel/PHP.
- **Base de datos**: MySQL 8 (vía Eloquent).
- **Almacén de sesiones** server-side.
- **Apache**: borde de mismo origen / proxy.

Diagrama general.

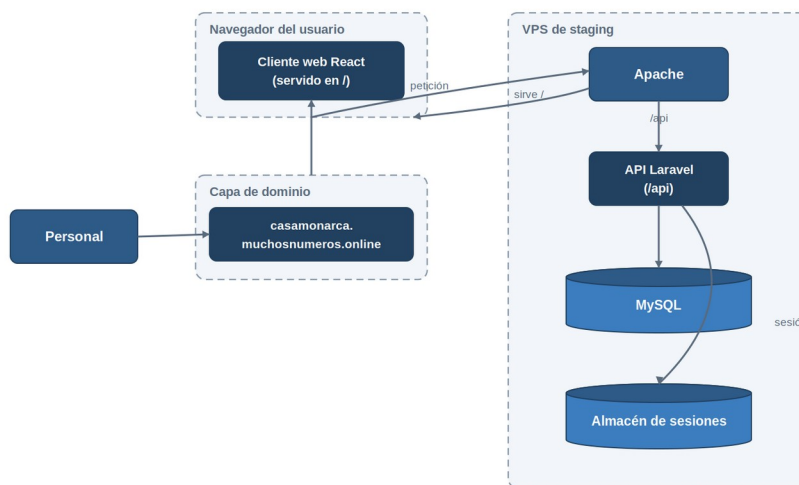


Fig. 1. Diagrama de contenedores: arquitectura de mismo origen (elaboración propia).

3. Stack Tecnológico

Lenguajes. PHP 8.3 (backend), TypeScript/JavaScript (frontend), SQL (MySQL).

Frameworks. Laravel 13 (backend, ORM Eloquent), React 19 + Vite (frontend).

Bibliotecas y APIs clave.

- *Frontend*: Web Crypto API nativa (verificación de firmas), qrcode (QR de TOTP), Vitest (unit), Playwright (QA visual).
- *Backend*: Encrypter de Laravel (AES-256-CBC), OpenSSL (openssl_verify), librerías de WebAuthn/FIDO2 y TOTP.
- *Infraestructura*: Apache, Git, scripts .sh de despliegue.

4. Estructura del Proyecto

Organización general (4 repositorios — org `MA2006B-601-Team-3`).

- `Access-Control-CM-API` — backend / API (principal).
- `Access-Control-CM-Web` — frontend / web (principal).

- evidencias_reto — documentación y entregables (diagramas, manual, esquema BD).
- Issue_Holder — seguimiento de issues.

Backend (`Access-Control-CM-API/app`).

```

app/
├── Enums/           # AuditEventType, AuditEventOutcome, UserRole, UserStatus
├── Http/
│   └── Controllers/Api/{Auth, Admin, Audit, Documents} # + HealthController
├── Models/          # User, Document, DocumentRevision, DocumentSignature,
                    # DocumentSignatureRequirement, DocumentTombstone,
                    # AuditEvent, AccountInvite, SecurityChallengeIntent,
                    # WebauthnCredencial, UserDevice
├── Providers/
├── Services/        # Auth, Admin, Audit, Documents, Security (lógica de negocio)
├── routes/api.php   # definición de endpoints + middleware
└── database/migrations # esquema MySQL

```

Frontend (`Access-Control-CM-Web/src`).

```

src/
├── pages/           # LoginPage, RegisterPage, ResetPasswordPage, SessionPage,
                    # AppShellPage, DocumentUploadPage, DocumentsPage,
                    # InviteManagementPage, AdminPage, LoggingPage, ForbiddenPage
├── components/      # auth/, ui/
├── lib/             # api.ts, auth.ts, csrf.ts, documents.ts, accountInvites.ts,
                    # adminUsers.ts, auditEvents.ts, securityChallenges.ts,
                    # webauthn.ts, localDocumentVerification.ts, signatureValidity.ts
├── config/
└── test/

```

Descripción de módulos clave.

- lib/api.ts — capa central de llamadas fetch (con credentials: include + CSRF).
- lib/webauthn.ts — registro/login con passkeys; lib/localDocumentVerification.ts — verificación local de firmas con Web Crypto API.
- Services del backend — una carpeta por dominio (Auth, Documents, Admin, Audit, Security).

5. Configuración del Entorno

Requisitos (versiones).

- *Backend*: PHP 8.3, Composer 2, MySQL 8.0, Git. Extensiones PHP: bcmath, ctype, curl, fileinfo, json, mbstring, openssl, pdo, pdo_mysql, tokenizer, xml, xmlwriter, zip.
- *Frontend*: Node.js 24, npm 11, Git.

Variables de entorno (backend `.env`).

```

env
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=casamonarca_api
DB_USERNAME=casamonarca
DB_PASSWORD=<contraseña-local-fuerte>
# APP_KEY se genera con php artisan key:generate (clave AES de Laravel)

```

Variables de entorno (frontend `.env`).

```

env

```

6. Instalación y Ejecución

```
bash
composer install
cp .env.example .env
php artisan key:generate
# crear BD y usuario MySQL (ver README del API), luego:
php artisan migrate
php artisan serve                # http://127.0.0.1:8000
```

```
bash
npm install
cp .env.example .env
npm run dev # http://localhost:5173
```

7. Base de Datos

```

classDiagram
    class sessions {
        id : string
        user_id : bigint
    }
    class password_reset_tokens {
        email : string
    }
    class cache {
        key : string
    }
    class users {
        id : bigint
        name : string
        email : string
        role : string
        status : string
        suspended_by_user_id : bigint
    }
    class sessions {
        id : bigint
        actor_user_id : bigint
    }
    class audit_events {
        id : bigint
        actor_user_id : bigint
    }
    class account_mutes {
        id : bigint
        muted_by_user_id : bigint
        verified_out_of_band_by_user_id : bigint
    }
    class security_challenge_intents {
        id : bigint
        actor_user_id : bigint
    }
    class cache_locks {
        key : string
    }
    class jobs {
        id : bigint
    }
    class job_batches {
        id : string
    }
    class webauthn_credentials {
        id : bigint
        user_id : bigint
        credential_id : string
    }
    class user_devices {
        id : bigint
        user_id : bigint
    }
    class documents {
        id : bigint
        owner_user_id : bigint
        uploaded_by_user_id : bigint
        current_revision_id : bigint
        approved_by_user_id : bigint
        title : string
        status : string
    }
    class document_revisions {
        id : bigint
        document_id : bigint
        parent_revision_id : bigint
        created_by_user_id : bigint
        revision_number : integer
        sha256 : string
    }
    class document_signatures {
        id : bigint
        document_revision_id : bigint
        signed_by_user_id : bigint
    }
    class document_signature_requirements {
        id : bigint
    }
    class document_tombstones {
        id : bigint
        deleted_by_user_id : bigint
        original_document_id : bigint
    }
    class failed_jobs {
        id : bigint
        uuid : string
    }

    sessions --> password_reset_tokens : has
    sessions --> cache : has
    sessions --> users : has
    sessions --> sessions : verifies
    sessions --> audit_events : acts
    sessions --> account_mutes : initiates
    sessions --> security_challenge_intents : initiates
    cache --> cache_locks : has
    cache --> jobs : has
    cache --> job_batches : has
    cache --> webauthn_credentials : has
    cache --> user_devices : has
    users --> users : suspends
    users --> sessions : initiates
    users --> sessions : verifies
    users --> audit_events : acts
    users --> account_mutes : initiates
    users --> security_challenge_intents : initiates
    users --> documents : creates
    users --> documents : approves
    users --> documents : signs
    users --> documents : assigns
    users --> document_revisions : has
    users --> document_signatures : has
    users --> document_signature_requirements : has
    users --> document_tombstones : deletes
    documents --> documents : updates
    documents --> document_revisions : has
    documents --> document_signatures : has
    document_revisions --> document_revisions : has
    document_revisions --> document_signatures : has
    document_signatures --> document_signature_requirements : fulfills
    document_signature_requirements --> document_signatures : requires
    document_tombstones --> documents : deletes
    failed_jobs --> failed_jobs : fails
  
```

Página 6 de 16

Tabla	Descripción
users	Usuarios: rol, estado, contraseña (hash), secreto 2FA (cifrado).
webauthn_credentials	Llaves públicas y metadatos de passkeys.
user_devices	Dispositivos asociados a usuarios.
documents	Documentos (dueño, quién subió, estado, confidencialidad).
document_revisions	Versiones (revisión padre, hash SHA-256, archivo, autor).
document_signatures	Firmas (quién, cuándo, tipo, hash, metadatos).
document_signature_requirements	Requisitos de firma (secuencia de firmantes).
document_tombstones	"Lápidas" de documentos borrados (metadatos sin contenido).
audit_events	Bitácora append-only (~49 tipos de evento).
account_invites	Invitaciones (token con hash, expiración, verificación).
security_challenge_intents	Intenciones de acciones sensibles (challenge, estado).
sessions	Sesiones server-side.

Relaciones (resumen). Un user tiene N webauthn_credentials y N documents (como dueño). Un document tiene N document_revisions (cadena padre→hijo) y N document_signatures; las document_signature_requirements definen quién debe firmar. Toda acción relevante genera audit_events.

8. API / Interfaces (núcleo del SDK)

API REST con autenticación por **sesión server-side + cookie + CSRF**. Las operaciones que mutan estado (POST/PUT/DELETE) requieren el header X-CSRF-TOKEN (obtenido de GET /api/csrf-token) y envían cookies (credentials: include). Las acciones sensibles (firmar, cambiar rol, borrar) usan un patrón de dos pasos ``/options` → `/verify`` con un *passkey challenge* fresco.

Convenciones. Base URL local: `http://127.0.0.1:8000/api`. Respuestas en JSON. Códigos: 200 OK, 202 requiere 2FA, 201 creado, 403 rol/permiso denegado, 419 CSRF inválido, 429 throttling.

8.1 Autenticación y sesión

Método	Ruta	Descripción
GET	/health	Verificación de disponibilidad.
GET	/csrf-token	Obtiene el token CSRF.
POST	/login	Login: 200 + usuario, o 202 requiresTwoFactor si hay TOTP.
POST	/login/totp	Completa login con código TOTP de 6 dígitos.
POST	/logout	Cierra sesión.
GET	/me	Usuario autenticado actual.
POST	/password-reset/complete	Completa restablecimiento (throttle 10/min).

8.2 Segundo factor (TOTP y WebAuthn)

Método	Ruta	Descripción
POST	/totp/enroll/options	Inicia enrolamiento TOTP (genera secreto/QR).
POST	/totp/enroll/verify	Verifica código y activa TOTP.
POST	/webauthn/register/options · /verify	Registra una passkey al usuario.
GET	/webauthn/credentials	Lista passkeys del usuario.
DELETE	/webauthn/credentials/{id}	Elimina una passkey.
POST	/webauthn/login/options · /verify	Login sin contraseña con passkey.
POST	/security-challenges/{intent}/cancel	Cancela un challenge de acción sensible.

8.3 Documentos

Método	Ruta	Rol mínimo	Descripción
POST	/documents	volunteer+	Sube documento (hasta 16 MB).
GET	/documents · /documents/{document}	non_coordinator +	Lista / detalle.
GET	/documents/{document}/download	non_coordinator +	Descarga revisión actual.
GET	/documents/{document}/revisions/{revision}/download	non_coordinator +	Descarga revisión #.
GET	/documents/{document}/verification	non_coordinator +	Estado de verificación.
GET	/documents/{document}/verification-bundle	non_coordinator +	Paquete para verificación local.
GET	/documents/{document}/verification-package	non_coordinator +	Paquete firmado (manifiesto RSA).
POST	/documents/{document}/revisions/options · /verify	coordinator+	Sube nueva revisión.
POST	/documents/{document}/revisions/{revision}/sign/options · /verify	coordinator+	Firma una revisión.
POST	/documents/{document}/delete/options · /verify	admin	Borra documento (deja tombstone).

8.4 Administración (rol admin, salvo invitaciones)

Método	Ruta	Descripción
GET	/admin/authorization-check	Sonda de autorización admin.
GET	/admin/users	Directorio de usuarios.
POST	/admin/users/{user}/role/options · /verify	Cambia rol (passkey admin).
POST	/admin/users/{user}/status/	Suspende/reactiva cuenta.

Método	Ruta	Descripción
	options · /verify	
POST	/admin/users/{user}/recovery/ options · /verify	Reset TOTP / revoca passkeys.
GET	/admin/document-approvals	Cola de aprobación de documentos.
POST	/admin/document-approvals/ {document}/approve · /reject	Aprueba/rechaza.
GET	/admin/signing-ledger	Libro centralizado de firmas.
GET	/audit-events	Bitácora de auditoría (filtrable).
GET/ POST	/admin/verification-package- signing-key (+ rotation/options · /verify)	Llave RSA de paquetes y su rotación.

8.5 Invitaciones (admin y coordinator)

Método	Ruta	Descripción
POST	/admin/invites	Crea borrador de invitación.
POST	/admin/invites/{invite}/ verify-out-of-band	Registra verificación fuera de banda.
POST	/admin/invites/{invite}/issue- link	Emite enlace y token de registro (una vez).
GET	/admin/invites	Lista invitaciones recientes.
POST	/admin/invites/{invite}/revoke	Revoca invitación pendiente.
GET	/invites/preview	Previsualiza una invitación (público, throttle 30/min).
POST	/invites/redeem	Redime invitación y crea cuenta (público, throttle 10/min).

Ejemplo de uso (flujo de login con CSRF).

```
bash
# 1) obtener token CSRF (guarda cookies)
curl -c cookies.txt http://127.0.0.1:8000/api/csrf-token
# 2) login enviando el token y las cookies
curl -b cookies.txt -c cookies.txt \
  -H "Content-Type: application/json" \
  -H "X-CSRF-TOKEN: <token>" \
  -d '{"email":"user@casamonarca.mx","password":"***"}' \
  http://127.0.0.1:8000/api/login
# Respuesta 200 (sin 2FA) o 202 {"requiresTwoFactor": true}
```

FIGURA — Petición / respuesta de la API (opcional)

Captura de Postman o cURL del flujo de login con CSRF
(ver ejemplo de la sección 8.5)

El equipo pega aquí la imagen final

Fig. Captura de petición/respuesta de la API (opcional — el equipo pega una captura de Postman/cURL).

8.6 Interfaz del frontend (SDK cliente)

La carpeta `src/lib/` es la interfaz programática que un desarrollador usa (o extiende) para hablar con la API sin reimplementar CSRF, cookies ni serialización. Toda llamada pasa por `apiFetch`, que inyecta `credentials: include` y `Accept: application/json`, ante un error HTTP, **lanza** `ApiRequestError` (con `status` y `errors` por campo). Las funciones que mutan estado piden el token CSRF internamente (`getCsrftoken`), por lo que el llamador no lo gestiona.

Patrón de acciones sensibles (firmar, cambiar rol, borrar): son dos pasos — `start...()` devuelve un `challenge` `WebAuthn (options)`; el navegador obtiene la `assertion` con `getWebauthnAssertion(options)`; y `verify...(assertion)` la confirma. Este patrón se repite en documentos, administración de usuarios y rotación de llaves.

8.6.1 `api.ts` — núcleo de transporte

Símbolo	Firma	Descripción
<code>apiFetch<T></code>	<code>(path: string, init?: RequestInit) => Promise<T></code>	Llamada base; añade cookies/headers y lanza <code>ApiRequestError</code> si <code>! response.ok</code> .
<code>buildApiUrl</code>	<code>(path: string) => string</code>	Construye la URL absoluta a partir de <code>apiBaseUrl</code> .
<code>getApiHealth</code>	<code>() => Promise<ApiHealthResponse></code>	Sonda GET <code>/health</code> .
<code>ApiRequestError</code>	<code>class extends Error { status: number; errors?: ApiFieldErrors }</code>	Error tipado de toda la capa.

8.6.2 `csrf.ts` y `auth.ts` — sesión y autenticación

Función	Firma	Descripción
<code>getCsrftoken</code>	<code>() => Promise<{ csrfToken: string }></code>	Obtiene el token CSRF (lo usan las mutaciones).
<code>login</code>	<code>(credentials: LoginCredentials) => Promise<LoginResponse></code>	Inicia sesión; respuesta de éxito o <code>{ requiresTwoFactor: true }</code> .
<code>completeTotpLogin</code>	<code>(credentials: TotpCredentials) => Promise<LoginSuccessResponse></code>	Completa login con código TOTP.
<code>startTotpEnrollment</code>	<code>() => Promise<TotpEnrollmentOptionsResponse></code>	Inicia enrolamiento TOTP (genera secreto/QR).
<code>verifyTotpEnrollment</code>	<code>(credentials: TotpCredentials) => Promise<TotpEnrollmentVerifyResponse></code>	Verifica el código y activa TOTP.
<code>completePasswordReset</code>	<code>(credentials: PasswordResetCredentials) => Promise<PasswordResetCompleteResponse></code>	Completa el restablecimiento de contraseña.

Función	Firma	Descripción
getCurrentUser	() => Promise<CurrentUserResponse>	Usuario autenticado (GET /me).
logout	() => Promise<LogoutResponse>	Cierra la sesión.
getFirstFieldError	(errors, field) => string undefined	Helper para mostrar el primer error de validación de un campo.

Passkeys (en `auth.ts`). `getWebauthnCredentials()`, `removeWebauthnCredential(id)`, `startWebauthnRegistration()` / `verifyWebauthnRegistration(payload)`, `startWebauthnLogin(email?)` / `verifyWebauthnLogin(payload)` — registro y login sin contraseña; las `start...` devuelven `options`, las `verify...` reciben la `assertion` serializada.

8.6.3 `webauthn.ts` — puente con el autenticador del dispositivo

Función	Firma	Descripción
getWebauthnAssertion	(options: WebauthnLoginOptions) => Promise<WebauthnLoginAssertionPayload>	Invoca <code>navigator.credentials.get()</code> , valida contexto seguro y serializa la firma a Base64-URL. Es el paso intermedio de todo flujo sensible.
isIpHostname	(hostname: string) => boolean	Detecta orígenes con IP cruda (WebAuthn los rechaza).

8.6.4 `documents.ts` — gestión documental

Función	Firma	Descripción
uploadDocument	({ file: File, title?: string }) => Promise<DocumentStoreResponse>	Sube un documento (multipart).
getDocuments	() => Promise<DocumentIndexResponse>	Lista de documentos accesibles.
getDocument	(documentId: number) => Promise<DocumentShowResponse>	Detalle con revisiones y firmas.
startDocumentRevisionUpdate	(documentId, { originalFileName, sha256, sizeBytes }) => Promise<...OptionsResponse>	Paso 1: pide challenge para subir nueva revisión.
verifyDocumentRevisionUpdate	(documentId, { assertion, file }) => Promise<DocumentStoreResponse>	Paso 2: confirma y guarda la nueva revisión.
startDocumentRevisionSign / verifyDocumentRevisionSign	(documentId, revisionId[, assertion]) => Promise<...>	Firma una revisión específica (patrón <code>options→verify</code>).
startDocumentSign / verifyDocumentSign	(documentId[, assertion]) => Promise<DocumentSignResponse>	Firma la revisión actual del documento.

Función	Firma	Descripción
startDocumentDelete/ verifyDocumentDelete	(documentId[, assertion]) => Promise<DocumentDeleteResponse>	Borra el documento (deja tombstone).
getDocumentVerification	(documentId) => Promise<DocumentVerificationResponse>	Estado de verificación del documento.
getDocumentVerificationBundle	(documentId) => Promise<DocumentVerificationBundleResponse>	Paquete (hash + firmas + llaves) para verificación local.
downloadDocumentBinary	(documentId) => Promise<ArrayBuffer>	Descarga el binario actual.
getDocumentDownloadUrl / ... RevisionDownloadUrl / ... VerificationPackageUrl	(...ids) => string	Construyen URLs directas (para enlaces de descarga).

8.6.5 Otros módulos

Módulo	Funciones principales	Para qué sirve
accountInvites.ts	getAccountInvites(limit?), createAccountInvite(payload), verifyAccountInviteOutOfBand(id, payload), issueAccountInviteLink(id, payload), revokeAccountInvite(id), previewInvite(token), redeemInvite(payload)	Ciclo completo de invitaciones (incluye redención pública).
adminUsers.ts	getAdminUsers(limit?), start/verifyAdminUserRoleUpdate, start/verifyAdminUserStatusUpdate, start/verifyAdminUserRecovery, getSigningLedger(), getDocumentApprovals(), approveDocument/rejectDocument, getVerificationPackageSigningKey()	Operaciones de administrador (todas las sensibles con options→verify).
auditEvents.ts	getAuditEvents(query)	Consulta filtrable de la bitácora (category, outcome, texto, fechas, paginación).
securityChallenges.ts	cancelSecurityChallenge(challengeIntentId)	Cancela un challenge de acción sensible pendiente.
localDocumentVerification.ts	verifyDocumentBundleLocally(bundle, fileBytes)	Verifica las firmas en el navegador con Web Crypto API; devuelve un reporte (fileHash, por firma verified + checks). Demuestra integridad sin confiar en el servidor.
signatureValidity.ts	getSignatureValidityState(expiresAt?, nowMs?)	Calcula estado/cuenta regresiva de validez de una firma para la UI.

8.6.6 Ejemplo de uso end-to-end (firmar una revisión)

```
ts
import { startDocumentRevisionSign, verifyDocumentRevisionSign } from "./lib/documents";
import { getWebauthnAssertion } from "./lib/webauthn";

// 1) El servidor emite un challenge ligado al hash de la revisión
const { options } = await startDocumentRevisionSign(documentId, revisionId);
// 2) El dispositivo (passkey/YubiKey) firma localmente; la clave privada nunca sale
const assertion = await getWebauthnAssertion(options);
// 3) El servidor verifica la firma con la clave pública y la registra
const result = await verifyDocumentRevisionSign(documentId, revisionId, assertion);
// -> result.signature contiene la firma persistida (no-repudio)

ts
// Verificación 100% local de un documento descargado (sin confiar en el servidor)
import { getDocumentVerificationBundle, downloadDocumentBinary } from "./lib/documents";
import { verifyDocumentBundleLocally } from "./lib/localDocumentVerification";

const { bundle } = await getDocumentVerificationBundle(documentId);
const fileBytes = await downloadDocumentBinary(documentId);
const report = await verifyDocumentBundleLocally(bundle, fileBytes);
// report.verified === true si el hash y todas las firmas cuadran
```

Nota de cobertura: la tabla documenta las funciones públicas de cada módulo de lib/. Si se agregan nuevas, mantener esta sección sincronizada con los export reales de Access-Control-CM-Web/src/lib/.

9. Flujos y Lógica de Negocio

Procesos clave.

- *Alta de usuario:* invitación → verificación fuera de banda → emisión de enlace → redención → enrolamiento 2FA.
- *Login:* CSRF → credenciales → (TOTP si aplica) → cookie de sesión.
- *Ciclo del documento:* subida → aprobación admin → revisiones (hash SHA-256, cadena padre→hijo) → firma → auditoría.
- *Firma digital:* challenge ligado al hash → firma local en el dispositivo (passkey) → verificación con clave pública en servidor → registro en document_signatures.

Reglas importantes.

- El enrolamiento de seguridad es obligatorio antes de módulos protegidos: coordinator requiere TOTP + passkey; non_coordinator/volunteer requieren TOTP.
- No se emite enlace de registro antes de la verificación fuera de banda.
- Acciones sensibles (firmar, cambiar rol, borrar) exigen passkey assertion fresca (/options → /verify).
- El token de invitación se entrega una sola vez; el servidor solo guarda su hash.

Casos críticos. Promoción a coordinator requiere que el usuario ya tenga passkey; cuentas admin bloqueadas para un flujo endurecido posterior; las invitaciones redimidas no se pueden revocar.

10. Ejemplos de Uso

- *Caso típico (voluntario)*: POST /documents con título y archivo → se crea documento + primera revisión con hash.
- *Caso típico (coordinator)*: subir nueva revisión (/revisions/options → /verify) y firmar (/sign/options → /verify).
- *Integración básica*: desde el frontend, todo pasa por lib/api.ts, que añade el header CSRF y credentials: include automáticamente.

11. Manejo de Errores

Estrategia. Respuestas JSON con código HTTP semántico y, en errores de autorización, un objeto error con código legible.

Mensajes comunes.

```
json
// 403 – rol no autorizado
{ "message": "Forbidden.", "error": { "code": "forbidden_role",
  "requiredRoles": ["admin"], "currentRole": "coordinator" } }
```

Otros: 419 (token CSRF ausente/expirado), 429 (throttling en login, invitaciones, recovery), 202 (requiere segundo factor).

Registro (logs). Toda acción relevante se registra en audit_events con actor, tipo, recurso, resultado (success/denied/failure), request_id, IP, user-agent, hash del ID de sesión y timestamp UTC.

12. Pruebas

Tipos de tests.

- *Backend*: feature tests con PHPUnit/Laravel contra un esquema MySQL dedicado (casamonarca_api_test); el bootstrap se niega a correr si la BD no parece de prueba.
- *Frontend*: unit tests con Vitest (p. ej. localDocumentVerification.test.ts, signatureValidity.test.ts) y QA visual con Playwright.

Cómo ejecutarlos.

```
bash
# Backend
cp .env.testing.example .env.testing # ajustar credenciales MySQL de prueba
php artisan test
# Frontend
npm run test
```

13. Despliegue

Cómo hacerlo. Scripts `scripts/deploy-staging.sh` en ambos repos.

- *Frontend:* sin runtime Node en el servidor; se compila `dist/` localmente, se sube un tarball con `scp`, se promueve en el VPS y se normalizan permisos (preserva `.htaccess` y `.well-known`).
- *Backend:* el script hace `git fetch/pull` remoto; los pasos de Laravel son manuales: `composer install --no-dev --optimize-autoloader`, `php artisan optimize:clear`, `php artisan migrate --force`. Normaliza permisos (dirs 755, archivos 644, `.env` 600, `artisan/scripts/*.sh` ejecutables).

Entornos. Desarrollo local → VPS de staging (`casamonarca.muchosnumeros.online`) → hosting compartido objetivo tipo HostGator.

Consideraciones. WebAuthn rechaza orígenes con IP cruda (usar dominio/localhost). El script no gestiona ownership ni etiquetas SELinux.

14. Seguridad

Autenticación. Sesión server-side + cookie `HttpOnly` + `CSRF`; doble factor TOTP y/o passkeys WebAuthn/FIDO2; contraseñas con `Argon2id/bcrypt`.

Manejo de datos sensibles. El secreto TOTP (`users.two_factor_secret`) se cifra con **AES-256-CBC** (llave `APP_KEY`, IV aleatorio, integridad por HMAC-SHA256 verificado antes de descifrar). Integridad de documentos/revisiones con **SHA-256**. Firmas con **ECDSA P-256 (ES256)** o **RSA PKCS#1 v1.5 + SHA-256 (RS256)**.

Buenas prácticas. No se implementa criptografía propia (Web Crypto API, OpenSSL, Encrypter de Laravel); archivos fuera del web root público; principio de menor privilegio por rol; bitácora append-only; verificación de assertion WebAuthn (RP ID hash, user presence, firma, sign counter).

Tabla 2. *Inventario de algoritmos criptográficos empleados (elaboración propia).*

Algoritmo	Tipo	Uso en el sistema
AES-256-CBC	Cifrado simétrico	Cifrar el secreto TOTP en la BD.
Argon2id / bcrypt	Hash de contraseñas	Guardar contraseñas de forma irreversible.
SHA-256	Función hash	Integridad de documentos/revisiones, hash de sesión, challenges.
HMAC-SHA1	MAC con llave	Generación del código TOTP (RFC 6238/4226).
HMAC-SHA256	MAC con llave	Integridad del payload cifrado de Laravel.
ECDSA P-256 (ES256)	Firma asimétrica	Firmar/verificar passkeys WebAuthn.
RSA PKCS#1 v1.5 + SHA-256 (RS256)	Firma asimétrica	Passkeys RSA y firma de paquetes de verificación.

15. Guía de Contribución

Cómo agregar funcionalidades. Backend: nueva ruta en `routes/api.php` → controller de acción única en `Http/Controllers/Api/...` → lógica en un Service → modelo Eloquent. Frontend: nueva función en `lib/` → componente/página en `pages/` o `components/`.

Antes de modificar. Clonar ambos repos, instalar dependencias, correr migraciones, levantar API + frontend y revisar `CONTEXT0-SISTEMA.md` y los READMEs.

16. Problemas Conocidos

- **Bugs actuales:** al ser primera versión, no se documentan bugs bloqueantes conocidos.
- **Limitaciones.**
 - VCS de documentos incompleto (eventos `vcs.*` definidos, VCS completo pendiente).
 - Bitácora append-only pero **no** encadenada criptográficamente (no es blockchain).
 - Atestación (attestation) de WebAuthn pendiente de endurecer.
 - Restricciones del hosting compartido tipo HostGator (sin servicios administrados, versiones de PHP/MySQL fijas, recursos limitados).

17. FAQ Técnica

- **El frontend no llega a la API en local.** Verificar que Laravel corre en `127.0.0.1:8000` y que el proxy de Vite apunta ahí (`API_PROXY_TARGET`).
- **WebAuthn falla al registrar.** Usar `http://localhost:5173` (no `127.0.0.1`); muchos proveedores de passkeys exigen un RP ID tipo dominio. Requiere contexto seguro.
- **419 al hacer POST.** Falta el header `X-CSRF-TOKEN` o expiró; volver a pedir `GET /api/csrf-token`.
- **Los tests no corren.** Crear `casamonarca_api_test` y `.env.testing`; el bootstrap se niega si la BD no parece de prueba.
- **`php artisan migrate` falla.** Revisar credenciales `.env` y que el servicio MySQL esté arriba.

18. Referencias

(Dejar solo enlaces con valor práctico para instalación/uso.)

- HostGator — matriz de software (versiones de PHP/MySQL soportadas).
<https://soporte.hostgator.mx/hc/es-419/articles/28443364586259>
- Laravel (documentación oficial), React + Vite (documentación oficial).
- FIDO Alliance / W3C WebAuthn; IETF RFC 6238 (TOTP).